CSC 251 Lecture 10: Intro to Data Science, Vectorization, NumPy Arrays

Introduction to Data Science



"What is Data Science?"

https://medium.com/@tejaswini.nareshit/important-topics-for-data-science-n-b7c30f93d482

New Workflow

Jupyter Notebook – human-readable documents containing code and rich text elements,
individual cells can be run to explore data and experiment

"Coding" versus "scripting"

IDEs:

- Visual Studio Code: https://code.visualstudio.com/docs/python/environments with Jupyter extension
- Install Anaconda distribution of Python (or Miniconda) for access to all packages
- Others: JupyterLab, Google Colab, PyCharm, Spyder: https://jupyter.org/

Tips:
Create a new environment (virtual/venv or conda) to avoid package conflicts

- With pip:

```
python -m venv myenv
source myenv/bin/activate
pip3 install numpy
pip show numpy
```

https://pip.pypa.io/en/stable/installation/

- With conda:

```
conda create --name myenv python=3.13
conda install -n myenv numpy
conda activate myenv
conda list -n myenv
```

https://www.anaconda.com/docs/getting-started/working-with-conda/environments

Package Installation Troubleshooting

Update Python
Restart VSCode
Check path (default IDE path?)
Create and install within a new environment (see instructions for your OS)

Numeric and Scientific Computing with NumPy

NumPy - Numerical Python with multi-dimensional array options, linear algebra, random number generation, Fourier transforms

Importance:

- Computational foundation for pandas and SciPy packages
- Vectorization - performs complex computations to an entire array **without** Python loops (10-100 times faster)
- NumPy arrays are stored as contiguous blocks of memory and use less memory than Python sequences.
  Recall, Python is generally dynamically-typed, where each data item is an object that contains:
  - the reference count for memory allocation and deallocation
  - a type
  - a size
  - a reference/pointer to a value

For non-homogeneous data, all this information must be stored for each item of the list. So, a list is comprised of a pointer to a block of pointers which each pointer points to a Python object (or cleverly-disguised C structure).

To import into your notebook:

```
import numpy as np
```

then use `np.` to access NumPy operations

Bookmark a favorite NumPy cheat sheet
https://intellipaat.com/blog/wp-content/uploads/2022/10/Python-Numpy-Cheat-Sheet-2022.pdf

https://images.datacamp.com/image/upload/v1676302459/Marketing/Blog/Numpy_Cheat_Sheet.pdf

Also: https://jakevdp.github.io/PythonDataScienceHandbook/index.html

Creating ndarrays

ndarray – flexible container for numerical data of a uniform type, an extension of the Python `array` module

See Jupyter notebook for output
Notice the difference between implicit display (last expression in a cell) and explicit printing

Example 1a: Using the array function, create a NumPy array from a list:

```
arr1 = np.array([3, 2.6, 8, 9.2])
```

constructor

iterable

Example 1b: Or from a list of equal-length lists:

```
arr2 = np.array([[1, 3, 5], [2, 4, 6]])
```

Example 1c: Or from a tuple: ← any iterable sequence

```
arr3 = np.array((10, 20, 30))
```

Example 1d: Or randomly generate the array:

```
arr4 = np.random.randn(3, 2)
```

```
array( [[ 0.79830299, -0.06713004],
        [ 0.82124466, -0.64234358],
        [-0.44777421, -1.58426862]])
```

Example 1e: Or use np functions to create an array of zeros or ones:

```
arr5 = np.zeros(8)
```

```
[0., 0., 0., 0., 0., 0., 0., 0.]
```

↑ floats by default

```
arr6 = np.ones(4)
```

```
[1., 1., 1., 1.]
```

Example 1f: Or an array of ones with the same length/shape and type:

```
arr7 = np.ones_like(arr4)
```

```
[[1., 1.],
 [1., 1.],
 [1., 1.]]
```

↑ using size/shape of this array

Example 1g: Or use a version of the Python range function:

```
arr8 = np.arange(2, 16, 3)
```

```
[ 2  5  8 11 14 17 20 23]
```

↑ no longer restricted to ints

NumPy Datatypes

If an `ndarray` is homogeneous, what's its data type?

Example 2a: You can check the shape and type of an array:

```
print(arr4.shape, arr4.dtype)
```

```
(3, 2) float64
```

default

Example 2b: You can specify the type of an array upon creation:

```
arr9 = np.array([5.4, 10.9, 15.2], dtype = np.float64)
print(arr9.shape, arr9.dtype)
```

```
(3,) float64
```

Example 2c: You can cast an array of one type to create another:

```
arr10 = arr9.astype(np.int32)
print(arr10, arr10.dtype)
```

```
[ 5 10 15] int32
```

Data type	Description
`bool_`	Boolean (True or False) stored as a byte
`int_`	Default integer type (same as C `long`; normally either `int64` or `int32`)
`intc`	Identical to C `int` (normally `int32` or `int64`)
`intp`	Integer used for indexing (same as C `ssize_t`; normally either `int32` or `int64`)
`int8`	Byte (-128 to 127)
`int16`	Integer (-32768 to 32767)
`int32`	Integer (-2147483648 to 2147483647)
`int64`	Integer (-9223372036854775808 to 9223372036854775807)
`uint8`	Unsigned integer (0 to 255)
`uint16`	Unsigned integer (0 to 65535)
`uint32`	Unsigned integer (0 to 4294967295)
`uint64`	Unsigned integer (0 to 18446744073709551615)
`float_`	Shorthand for `float64`.
`float16`	Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
`float32`	Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
`float64`	Double precision float: sign bit, 11 bits exponent, 52 bits mantissa
`complex_`	Shorthand for `complex128`.
`complex64`	Complex number, represented by two 32-bit floats
`complex128`	Complex number, represented by two 64-bit floats

https://jakevdp.github.io/PythonDataScienceHandbook/02.01-understanding-data-types.html

Note: these will often be abbreviated, for example, `'f8'` for `'float64'` or `'i4'` for `'int32'`.

Indexing ndarrays

1D and 2D arrays will be indexed as you've seen before:

```
x[10]          num[-3]          profit[2][4]          scores[-1][-3]
```

Slicing NumPy Arrays

Use the same list slicing convention for `ndarrays`:

```
array[start:stop:step]
```

which includes the start value but excludes the stop value. To access the last element in each row of a 2D array:

```
array[:,-1]
```

last column

all rows

Example 3: Array slices follow the same rules for list slices but are VIEWS on the original arrays. Slicing a list produces a sublist which is a deep copy. Array slices are not copies but rather reference the original array, so modifications are seen in the original.

```
orig = np.array(range(0,100,10))

print(orig)                          [ 0 10 20 30 40 50 60 70 80 90]
orig                                 array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])

s1 = orig[2:5]
s1                                   array([20, 30, 40]

s1[0] = 2000
s1                                   array([2000,   30,   40])

orig                                 array([ 0, 10, 2000, 30, 40, 50, 60, 70, 80, 90]))
```

A view or shallow copy!

original also changed!

Changes can be broadcasted through the slice:

```
orig[5:] = 0
print(orig)                          array([  0,   10, 2000,  30,  40,  0,  0,  0,  0,  0])
```

To create a deep copy:

```
s2 = orig.copy()

s3 = orig[1:5].copy()
```

array or slice

Example 4: For multidimensional arrays, omitting later indexes will return a lower dimensional object.

```
three = np.array([[[1,2,3],[4,5,6]], [[7, 8, 9],[10,11,12]]])
print(three)

[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
```

```
print(three[0])
```

```
[[[1 2 3]
  [4 5 6]]
```

```
three[1] = 0
print(three)
```

broadcast through "element"

```
[[[1 2 3]
  [4 5 6]]
```

Example 5: Slicing multidimensional arrays:

```
two = np.array([[10,20,30],[40,50,60], [70, 80, 90],[100,110,120]])
print(two)
```

```
[[ 10  20  30]
 [ 40  50  60]
 [ 70  80  90]
 [100 110 120]]
```

```
s4 = two[:2]
```

```
[[10 20 30]
 [40 50 60]]
```

```
s5 = two[:, :1]
```

```
[[ 10]
 [ 40]
 [ 70]
 [100]]
```

```
s6 = two[2:, 1:2]
```

?

NumPy Vectorized Operations

Vectorization allows for batch operations on data without looping, accomplished through NumPy ufuncs (universal function) for unary and binary operations

Operator	Equivalent ufunc	Description
`+`	`np.add`	Addition (e.g., `1 + 1 = 2`)
`-`	`np.subtract`	Subtraction (e.g., `3 - 2 = 1`)
`-`	`np.negative`	Unary negation (e.g., `-2`)
`*`	`np.multiply`	Multiplication (e.g., `2 * 3 = 6`)
`/`	`np.divide`	Division (e.g., `3 / 2 = 1.5`)
`//`	`np.floor_divide`	Floor division (e.g., `3 // 2 = 1`)
`**`	`np.power`	Exponentiation (e.g., `2 ** 3 = 8`)
`%`	`np.mod`	Modulus/remainder (e.g., `9 % 4 = 1`)

Also available:

- matrix operations
- trig functions
- absolute value
- logs and exponentials
- bitwise arithmetic operators
- comparison operators

Example 6: Vector addition

```
arr11 = arr4 + arr7
```

← easier to remember, more intuitive

```
[[-0.68070219,  2.11985439],
 [-0.12816789, -0.06451319],
 [-0.25987858,  0.48863398]]
```

which is a wrapper for the add function:

```
arr12 = np.add(arr4, arr7)
```

Example 7: Member-wise comparison

```
com = arr4 > arr7
```

```
[[False,  True],
 [False, False],
 [False, False]]
```

More specialized functions in SciPy which offers integration, optimization, interpolation, image and signal processing, stats, and mathematical function.

NumPy Meshgrid Plots

The NumPy library has a way to evaluate scalar and vector fields over n-dimensional grids.
Start with 1D arrays of coordinates:

```
x = np.arange(-1,1,0.1)
y = np.arange(-1,1,0.1)
```

see notebook to output x,y,z

The results of meshgrid is a coordinate grid. The xx and yy arrays are 2D arrays containing x- and y-coordinates. they can be used in calculation and matrix operations. The 2D array zz contains the scalar values of the point-by-point calculations :

```
xx, yy = np.meshgrid(x,y)
zz = xx * yy
```

The values can best be visualized on a heat map or contour plot that shows the scale and a corresponding colorbar:

```
import matplotlib.pyplot as plt
plt.contourf(x,y,zz)
plt.axis('scaled')
plt.colorbar()
plt.title("Contour plot of x * y")
```

← more on this later

